# Mini-project 2 in Advanced Machine Learning (02460)

by Frederik Svane (s224766), Marcus Christoffersen (s224750),
Nicholas Larsen (s224175) & Thor Højhus (s224233) on 01 April 2026

## Part A: Pull-back geodesics

We train a VAE (Kingma and Welling, 2014) with a standard Gaussian prior and a Gaussian likelihood on a 3-class subset of MNIST (2048 observations). The latent space is two-dimensional to permit visualisation. Encoder and decoder are symmetric convolutional networks; Gaussian noise ($\sigma = 0.05$) is added to training inputs to improve decoder smoothness.

The pull-back metric (Arvanitidis et al., 2018) induced by the decoder mean $f\colon \mathcal{Z} \to \mathcal{X}$ equips the latent space with a Riemannian structure: the metric tensor at $z$ is $G(z) = J_f(z)^\top J_f(z)$, where $J_f$ is the decoder Jacobian. Geodesics are shortest paths under this metric; we approximate them by discretising a curve $c(t)$ into $N = 20$ points and minimising the energy $\mathcal{E}(c) = \sum_i \|f(c(t_i)) - f(c(t_{i+1}))\|^2$ over the interior points with Adam (lr $= 10^{-2}$, 500 steps), keeping the endpoints fixed.

Figure 1 shows the encoded test set coloured by digit class alongside 25 geodesics between random latent pairs. The geodesics visibly curve away from low-density regions between clusters: paths connecting points in different clusters bend to follow the data manifold rather than cutting through empty space. Within a single cluster the geodesics are nearly straight, as expected when the decoder varies smoothly. Across independent training runs the qualitative bending pattern persists, but the precise paths shift because latent representations are only identified up to rotation and translation. Consequently, individual geodesic coordinates are not directly comparable across runs, motivating the ensemble approach in Part B.

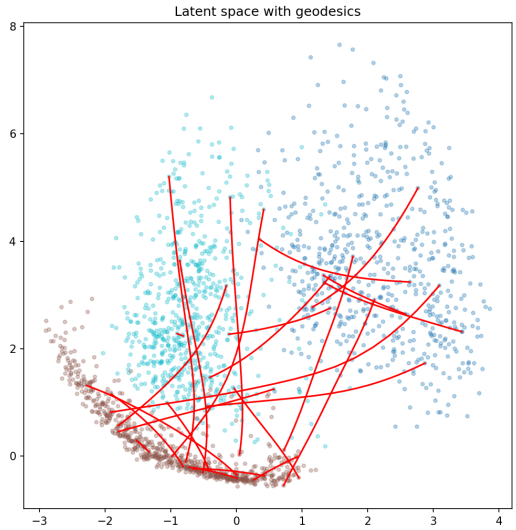


Figure 1: Latent space with 25 pull-back geodesics (red) overlaid on the encoded test set (coloured by digit class). Geodesics curve around low-density regions between clusters.

## Part B: Ensemble VAE geometry

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

# Code snippets

## 1. Pull-back curve energy

The curve energy under the pull-back metric is computed by mapping each point on the discretised latent curve through the decoder mean and summing squared differences in data space.

```
def curve_energy(curve, decoder):
    decoded = decoder(curve).mean
    diffs = decoded[1:] - decoded[:-1]
    return (diffs ** 2).sum()
```

Listing 1: Curve energy (ensemble_vae.py)

## 2. Geodesic computation via energy minimisation

A piecewise-linear curve is initialised between two latent endpoints. Only the interior points are optimised; the endpoints stay fixed.

```
def compute_geodesic(a, b, decoder, num_points=20,
                     num_epochs=500, lr=1e-2):
    t = torch.linspace(0, 1, num_points,
                       device=a.device).unsqueeze(1)
    curve = (1 - t) * a + t * b  # straight-line init

    interior = curve[1:-1].clone().detach().requires_grad_(True)
    optimizer = torch.optim.Adam([interior], lr=lr)

    for epoch in range(num_epochs):
        optimizer.zero_grad()
        full_curve = torch.cat(
            [a.unsqueeze(0), interior, b.unsqueeze(0)], dim=0)
        energy = curve_energy(full_curve, decoder)
        energy.backward()
        optimizer.step()

    with torch.no_grad():
        final_curve = torch.cat(
            [a.unsqueeze(0), interior, b.unsqueeze(0)], dim=0)
    return final_curve
```

Listing 2: Geodesic optimisation (ensemble_vae.py)

# References

G. Arvanitidis, L. K. Hansen, and S. Hauberg. Latent space oddity: on the curvature of deep generative models. In *International Conference on Learning Representations*, 2018.

D. P. Kingma and M. Welling. Auto-encoding variational Bayes. In *International Conference on Learning Representations*, 2014.